BRAVO PRODUCTION FINDINGS · 01 OF 05 · PEOPLE

Is Bravo actually faster to build in?

> *"Developing in Lora is hard, but developing in Bravo is faster and easier because the team now knows what to do."*

0	**1.27×**	**2d**	**5→1**
ISSUES ASSIGNED ON THE TWO BOARDS THE CLAIM CITES	**BRAVO'S** TICKET VOLUME OVER LORA'S, JAN–AUG — **INVERTED** FROM 1.83× THE OTHER WAY WHEN THE MISSING `LN` PROJECT WAS COUNTED	AUGUST MEDIAN CYCLE TIME — ON BOTH PLATFORMS	REPOSITORIES PER CHANGE — THE REAL EFFECT

Audience Engineering managers, tech leads, CTO office **Sources** Jira DF · D2W · BL · BLCS **Read** 2026-09-10

Six claims, graded against the data

The DF4W and DF2W boards show Bravo development moving faster	UNFALSIFIABLE	`DF` holds 24 issues, 23 of them Epics. `D2W` holds 22, all Epics. Every one unassigned, none resolved. These are product intake backlogs. Nothing on them can be timed.
Bravo development is faster	NOT SUPPORTED	Bravo's **two** delivery projects (`BLCS` + `LN`) created **≈6,518** issues Jan–Aug against LORA's ≈5,144 — **Bravo 1.27×**. That *reverses* the earlier 1.83× LORA finding, which counted only `BLCS` . But ticket volume measures decomposition as much as output, so it **retires the metric** rather than supporting the claim. August median created→resolved is **2 days on both**; in June LORA was *faster*.
...because the team now knows what to do	CONFOUNDED	Bravo's team has worked on this codebase since 2022 — 94 all-time authors, 23 active in 2026. That is tenure on a four-year-old codebase, not a platform being easier to learn.
Developing in Bravo is *easier*	SUPPORTED	One change is fewer artefacts. A Bravo ticket decomposes into `[BE]` / `[FE]` / `[QA]` ; the LORA equivalent into five repositories and five deploys. Real, measurable, and the defensible core of the claim.
Bravo has a shallower onboarding curve	HALF TRUE	Supported on skills — Java/Spring/Camunda are commodity. Unsupported on codebase: a new hire reads a 498k-LOC monolith with product identity in five places. The advantage is spent by week three.
DF2W is a live Bravo product delivering faster than LORA	REFUTED	**Corrected 2026-09-10:** DF2W is **not live at all** — it is in **UAT with an LOS penetration test running**, go-live epics still open. This pack had called it "fully configured in production", which reads as released-and-unused; it is pre-release. It has started **zero** applications in 90 days. DF4W carries **5.5%** of Bravo's volume. The two cited products are the smallest thing Bravo runs — and one is not running.

THE THROUGH-LINE

The claim is asking about ergonomics and being defended with velocity.

On velocity the data does not support it. On ergonomics it is right — and nobody had measured the size of the effect until now: five repositories per change against two lanes.

WHAT THE DATA REFUSES

Every throughput measure in this document favours LORA or ties. Bravo's June median cycle time is **9 days against LORA's 2**, and the cause is release batching, not architecture.

WHAT THE DATA CONFIRMS

One integration change is **1 repository and 1 deploy** in Bravo against **5 and 5** in LORA, in a mandated order. That is the whole real effect, and it is worth taking seriously.

The two boards the claim cites contain no development work

PROJECT	BOARD	ISSUES	TYPES	ASSIGNED	RESOLVED	FIRST → LAST CREATED
DF (DF4W)	2099, kanban	**24**	23 Epic, 1 Task	0	0	2025-12-12 → 2026-09-09
D2W (DF2W)	2877, kanban	**22** of 36 keys	22 Epic	0	0	2026-06-02 → 2026-08-28
BL (LORA DP)	683	**10,138** keys	Story, Sub-task, Task, Bug	assigned	thousands	2024-11-21 → 2026-09-10

Twelve of the 24 `DF` issues were created inside **four minutes** on 2025-12-12 — a backlog-loading session, not a sprint. `D2W`'s first sixteen were created inside **seventeen minutes**. Nine months after the first DF epic was written, not one has been assigned to anyone or moved out of To Do.

NOT A CRITICISM OF THE BOARDS

A product intake backlog is a legitimate artefact and both are well-formed ones. The point is what can be concluded from them: **a timeline view of these boards shows product intent, not delivery.** Any side-by-side putting board 2099 next to board 683 compares an epic backlog with a ticket stream.

Bravo's development work lives in `BLCS` and `LN`

The comparison the claim wants has to be made somewhere else — and there are **two** such places, not one. Recent `BLCS` titles: *"[BE] Prepare Query Change GMB to NMH"*, *"[DF2W UW] Sync Surveyor Status Underwriting Return"*. Recent `LN` titles: *"[Surveyor] Revamp Filter Assignment Team Head Surveyor"*, *"[FE][TechDebt] Operation Route Validation based on Role"*. **Added 2026-09-10 after Team Bravo pointed out `LN` was missing — it is the larger of the two.**

	BLCS — BRAVO LOS	LN — BRAVO SURVEYOR & VERIFICATOR	BL — LORA DP
First issue	`BLCS-985`, 2023-12-19	`LN-76`, 2024-09-09	`BL-3`, 2024-11-21
Latest at read time	`BLCS-4811`, 2026-09-09	**`LN-6626`, 2026-09-10** created 14:35, resolved 17:43	`BL-10138`, 2026-09-10
Issue types in use	Epic, Story, Story Bug, Task, Sub-task, Bug	Epic, Story, Task, Sub-task	Epic, Story, Story Bug, Task, Sub-task, Bug
Sub-task convention	`[BE]` `[FE]` `[QA]` `[DF2W UW]`	`[Surveyor]` `[Operation]` `[FE]` `[FE][TechDebt]` `[Master]`	`[LSS]` `[LGS]` `[LPW]` `[LTW]` `[LTS]` `[LBOFE]` `[QA]` `[PR]`
Delivers into	`bravo-bpm-service`	**the three operator consoles** — 763,861 LOC, 829 tests	13 LORA repos

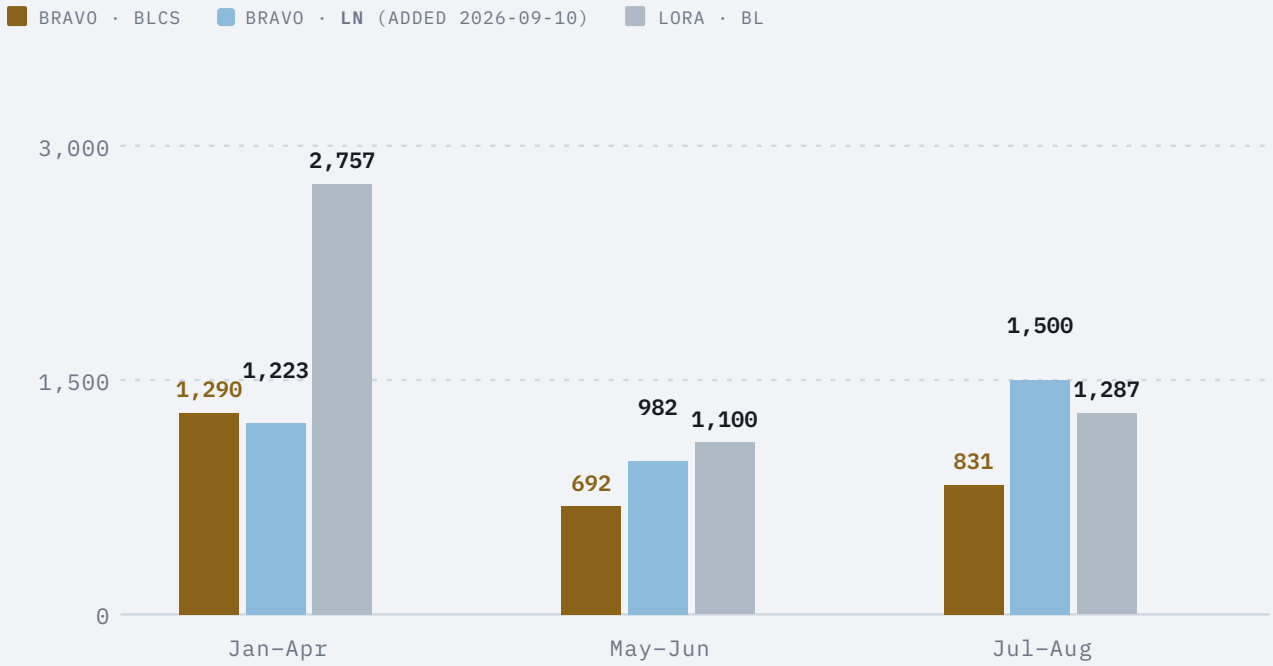
The taxonomies match, which is what makes the comparison possible — both projects run the same PMO template. Counting a `BLCS` ticket against a `BL` ticket is counting like against like at the *process* level. It is **not** like against like at the *change* level, which is where §6 matters.

DF2W WORK APPEARS IN *BOTH* DELIVERY PROJECTS

`BLCS-4420` "[DF2W Scoring] Penyesuaian sumber Data Downpayment" and `BLCS-4426` in one; `LN-6479` "[DF2W] E2E Testing", `LN-4533` "[DF2W] [O] Application Tracking" in the other. So DF2W **is** being actively built and tested across two squads. It is just not tracked on the DF2W intake board.

Its real status, verified 2026-09-10: UAT branch mapping seeded (`INS-6245`), go-live worker TAC closed (`TDF-4273`, 09-09), **LOS pen test live** (`ADI-1312`, `BLCS-4799`), pre-go-live epics open (`D2W-5`, `LN-4573`). **Pre-release, not abandoned.**

Throughput: with `LN` counted, the LORA lead reverses



Corrected 2026-09-10. With Bravo's second delivery project `LN` counted (**3,705** issues, larger than `BLCS`), Jan–Aug totals are Bravo **6,518**, LORA **5,144** — **Bravo 1.27×**, and **1.81×** in Jul–Aug. The previous reading of this slide — LORA ahead 1.83×, its lead narrowing from 3.2× to 1.5× — was an artefact of counting one of Bravo's two projects against all of LORA's.

METHOD

Jira issue keys are allocated monotonically per project, so the first key after a month boundary bounds the number created before it. Boundaries read directly: `BLCS-1939` / `LN-2843` / `BL-4795` at 1 Jan through `BLCS-4752` / `LN-6548` / `BL-9939` at 1 Sep. `LN` was read at four boundaries, so windows are pooled two months at a time and the other two projects pooled to match.

DOUBLE-COUNTING TESTED 2026-09-10 — THE PROJECTS DO NOT OVERLAP

If one change were a `BLCS` back-end story *and* an `LN` console story, summing would count it twice. Four tests say it is not: **(1)** `BLCS` is *Team Scoring & Underwriting*, `LN` is *Team Surveyor & Verificator* — split by **squad and domain, not tier**; **(2)** `LN` sub-tasks are `[BE]` 18 / `[FE]` 15 / `[QA]` 7 — full-stack, not a front-end half; **(3)** capability prefixes are **disjoint** (`[DF2W Scoring]`, `[UW 4W]` vs `[DF2W] [O]`, `[DF2W] [S]`); **(4) zero** cross-project issue links. **No pair of 80 sampled summaries describes the same change.** So the range collapses to its top end: **6,518**, **Bravo 1.27×**. Residual inflation is per-sprint QA written once per squad — a handful of tickets, not a doubling. Key deltas remain an **upper bound** for all three projects (`D2W` has **14 of 36 keys empty**).

WHAT THIS DOES NOT MEAN — AND THE CAUTION NOW CUTS BRAVO'S WAY

It does not mean Bravo ships 1.27× the product change. Ticket count measures **decomposition** at least as much as output. That argument was used to discount LORA's apparent lead; **it must now be used to discount Bravo's.** The right conclusion is that ticket volume cannot settle this in either direction. See §6.

Cycle time: no durable advantage either way

Created → `resolutiondate` in calendar days, for every resolved issue created in a matched window. Two windows, **six** samples, `n = 50` each, taken six months apart to avoid reading a single sprint. **§4 sampled `BLCS` and `BL` only until 2026-09-10 — `LN` was added to §3 and §5 but not here. Now measured; the finding gets stronger.**

SAMPLE	WINDOW	N	MEDIAN	MEAN	P90-ISH TAIL
Bravo BLCS	2026-06-01 → 06-08	50	**9 d**	22.3 d	60–98 d (9 issues)
Bravo LN	2026-06-01 → 06-02	50	**68 d**	63.4 d	40–101 d (**45 issues**) — a backlog load, not a cadence
LORA BL	2026-06-02 → 06-03	50	**2 d**	18.5 d	43–90 d (13 issues)
Bravo BLCS	2026-08-11 → 08-18	50	**2 d**	3.9 d	10–15 d (8 issues)
Bravo LN	2026-08-11 → 08-18	50	**1 d**	2.4 d	8–10 d (9 issues)
LORA BL	2026-08-11 → 08-12	50	**2 d**	5.3 d	14–29 d (9 issues)

1 · THE MEDIANS CONVERGE AT 1–2 DAYS

In August all three projects are **indistinguishable** — `LN` 1 d, `BLCS` 2 d, `BL` 2 d — and `LN`, Bravo's *larger* project, has the lowest median *and* mean. Whatever difficulty LORA imposes does not show up here; and adding Bravo's missing project opens no gap in Bravo's favour either.

2 · BOTH JUNE FIGURES ARE BACKLOG LOADS, NOT CADENCES

`LN`'s 68 d comes from **45 of 50 issues created in a 37-minute window** on 2026-06-01 21:19–21:56 — the `[DF2W] [O]` / `[S]` stories. That interval measures how long the **DF2W programme** took, not ticket turnaround; the 5 organically-created issues in that sample have a median of **0 d**. `BLCS`'s 9 d is the same artefact: nine Stories created 2026-06-01, batch-closed on release. **Both Bravo projects loaded DF2W backlog that day** — the programme kickoff, seen twice.

3 · SAME BIMODAL SHAPE ON BOTH

A mass of sub-3-day sub-tasks and a tail of Stories that close on release. That is **one delivery process running on two platforms** — exactly what §2 predicted.

Limits. Each sample covers one to six working days — a snapshot of two release cadences, not a distribution over the year. Windows differ in calendar length because the projects create tickets at different rates — `LN` fastest of the three. **The June `LN` sample is a backlog load and is not a cadence at all; the August windows are the comparable ones.** All samples exclude unresolved issues, truncating the tail equally.

Team shape is a prior decision, not a platform property

THIS SECTION USED TO MAKE AN ARGUMENT. IT IS WITHDRAWN.

Earlier versions read *"Seven engineers is the whole Bravo LOS delivery capacity"* and called it **the strongest argument against placing new product families on Bravo.** That inference is wrong. **Bravo's headcount is the output of a previous CTO decision to consolidate engineering onto LORA** — not a property of Bravo, of BPMN, or of Camunda. The office that would decide where a new product family goes sets the allocation and can reverse it. **Citing the consequence of a decision as evidence about that decision is circular.**

THE MEASUREMENT STILL STANDS — IT IS THE INFERENCE THAT DOES NOT

DISTINCT ASSIGNEES, 100 SAMPLED ISSUES	BRAVO BLCS	LORA BL
Across both samples	**7**	**22**
June sample	6	19
August sample	7	13
Unassigned resolved issues	1	3

AND IT TAKES §3 DOWN WITH IT

§3 has now been overtaken twice. It explained LORA's **1.83×** as *decomposition*; headcount was a second explanation pointing the same way; and then the 1.83× turned out to be an artefact of the missing `LN` project, leaving **Bravo ahead at 1.27×**. The instruction is unchanged and now applies in Bravo's favour: no per-team velocity claim built on it is a platform measurement. **§6 survives**, because it counts artefacts per *change* rather than tickets per team.

WHAT SURVIVES — PLATFORM-NEUTRAL

In both samples essentially every `Story` is assigned to a **single account** while sub-tasks distribute across the rest — 9 of 9 long-lived Stories in June, 9 of 11 in August. Whoever holds the shape of a Bravo change today is a **bus factor of one**, on a service carrying ~94% of origination. That is a live operational risk needing an owner **whatever is decided about new products** — and it is an argument for neither platform. Whether LORA's spread survives a re-staffing is unknown.

AND ONE ROW GAINS WEIGHT: THE HIRING POOL

If capacity is to be added, the question becomes *which platform can BFI actually hire for?* Java 17 / Spring Boot / Camunda is a commodity skill set. Go plus a bespoke GSM planner plus Temporal is not — LORA's own people.md puts it as *"you can get Java or Golang developers, but they won't be able to immediately read the code."*

This is the one staffing-adjacent fact that is not CTO-reversible, because it is a property of the labour market rather than of an allocation decision — and it favours Bravo. Bounded in §7: the advantage is front-loaded and largely spent by week three.

"The team now knows what to do" is still confounded — but by tenure, not by size.

Bravo's assignees have been on this codebase for up to four years; LORA's include this year's joiners closing tickets at the same median. Knowing what to do is a property of people and their tenure, **and people can be reallocated.** It is not a property of either platform.

The shape of one change — this is what “easier” actually means

This is the finding that survives every caveat, and the one the claim is really about. Take a single ordinary change: **bumping an upstream integration to v2.** In LORA it appears as six tickets across five repositories, all created within two minutes on 2026-08-11, all assigned to one engineer.

```
BL-9528  [LSS]    add one-obligor-summary-check-v2 schema
BL-9529  [LGS]    add one-obligor-summary-check-v2 handler
BL-9530  [LPW]    migrate get-obligor-summary to v2
BL-9531  [LTS]    register v2 proxy in authorization
BL-9532  [LBOFE]  migrate auto-exposure component to v2
BL-9533  [PR]     Review: one-obligor-summary-check-v2 refactor
```

That is the Schema-First order — `LSS → LGS → LPW/LTW → LTS → LBOFE` — mandated by LORA’s own service-dependencies rule, rendered as a ticket each. **It is not overhead someone added; it is the architecture made visible in the tracker.** The pattern repeats: `BL-9506/9507/9508` split one fee calculation across `[LSS]`, `[LTW]` and `[LBOFE]`.

THE EQUIVALENT BRAVO CHANGE IS ONE SUB-TASK

```
BLCS-4433  [4W Scoring] BE Implementation –
           Adjustment NST Rate – RO Rate
           Standard Override Indicator
```

OR, WHEN THE FRONT END IS INVOLVED, TWO LANES PLUS QA

```
BLCS-4388  [FE] Development – DF2W section &
           field visibility on Ringkasan Data
BLCS-4389  [FE] Manual Testing – verify DF2W
           reduced view + 4W no-regression
BLCS-4390  [FE] Deploy – Deploy to SIT
```

One repository and one deploy, against five and five

PER ONE INTEGRATION CHANGE	BRAVO	LORA
Repositories touched	**1** — `bravo-bpm-service`	**5** — LSS, LGS, LPW, LTS, LBOFE
Deploys to land it	**1**	up to **5**, in a mandated order
Jira artefacts	1–3	5–6
Backward-compatibility obligation	none in-process	additive-only — 8 worker versions run concurrently
Cost when the change is wrong	one revert	a version-ordered unwind

> ### This is the entire defensible content of "faster and easier", and it is worth taking seriously.
>
> A monolith with a relational aggregate really is fewer moving parts per change than thirteen repositories with a schema-first ordering rule. It also explains §3 without any appeal to productivity: LORA's ticket count is ~1.8× Bravo's and its per-change ticket count is ~2–5× Bravo's — so per change, the two teams may well be shipping at similar rates with very different bookkeeping.

WHAT THE FIVE-REPO TAX BUYS

Product isolation at the data layer, schema-validated contracts, and the ability to add an automated step with **no orchestration edit at all** — 172 activities against 3 hard precursors.

WHAT THE ONE-REPO CHANGE PAYS

One job executor, one deployment, and a BPMN edit that **ships for every product at once.**

Neither is free. The claim is right that Bravo's per-change cost is lower. It is silent on what that cost buys.

The onboarding curve

	BRAVO	LORA
Hiring pool	Java 17 / Spring Boot / Camunda 7 — **commodity**	Go + a bespoke GSM planner + Temporal — *"you can get Java or Golang developers, but they won't be able to immediately read the code"*
Written onboarding	**none found in this analysis pack**	`docs/onboarding/` — 14 files, ~5,550 lines, with working hands-on tooling
Week-1 curriculum	**not established**	**also not established** — LORA's own people.md grades this *"overstated, but the entry point is inverted"*
What a hire must actually read	498k LOC main Java; product identity in 5 places; 197 `setStatus` sites in 73 files; a 10,419-line surveyor assignment service	~617k LOC Go across 13 repos; 172 activity constructors; 155 preconditions with no generated index
Concepts with no analogue elsewhere	BPMN escalation as a return channel; a per-application jsonb activity on/off matrix; five-place product discrimination	ReadSet/WriteSet planning; determinism constraints; schema-versioned task queues

The commodity-skills advantage is real and it is **front-loaded**. It gets a hire to their first compile faster. It does not help with the parts of Bravo that are actually hard, and by the evidence of this pack those are large: an aggregate root with no behaviour, a lifecycle smeared across four status vocabularies, and orchestration untested by every one of the 2,286 test files in the estate.

> **Neither platform has a week-1 curriculum. That is the finding both teams share — and it is cheaper to fix than either architecture.**

What the claim gets right, and what it does not

“Developing in Lora is hard”	PARTLY TRUE	And *specifically* true: five repositories, a mandated ordering rule, additive-only schema changes, eight concurrent worker versions. Measured in §6.
“Developing in Bravo is faster”	NOT SUPPORTED	The ticket measure **inverted** on a scope correction — LORA 1.83× became Bravo 1.27× — which retires it in both directions. What is left is identical August median cycle time and a slower June median. **No *reliable* throughput measure favours either platform.**
“...and easier”	SUPPORTED	One repository, one deploy, no cross-service ordering. This is the real effect and it is worth about **3–5 artefacts per change**.
“...because the team now knows what to do”	CONFOUNDED BY TENURE	Bravo’s seven active assignees have been on this codebase for up to four years. LORA’s twenty-two include this year’s joiners closing tickets at the same median. Knowing what to do is a property of the people, and **the seven do not scale.**

A FIFTH THING THE CLAIM DOES NOT SAY, AND SHOULD

The two products it cites are the two **smallest** things Bravo runs. DF4W is the only product on the unified spine — **5.5%** of started applications over 90 days. DF2W is fully configured and has taken **zero**. If those are the reference implementations for “Bravo is easier”, then the easiness has not yet been tested at volume, on the monoliths, where **~94% of the book actually runs** and where 2026 saw more commits than the spine did.

Six actions

4 **Name Bravo's bus factor** · S, URGENT

Seven active assignees, with the Story layer held by one account, on the service carrying ~94% of origination. Before any new product family is placed on Bravo, that number has to be a staffing decision someone has made on purpose.

1 **Stop citing boards 2099 and 2877 as delivery evidence** · S

Zero assigned, zero resolved. Either wire DF/D2W epics to their `BLCS` children so the timeline reflects delivery, or state explicitly that Bravo delivery is tracked in `BLCS` and compare there.

2 **Measure changes, not tickets** · S–M

This just proved itself: the ratio swung from LORA 1.83× to Bravo 1.27× on the addition of one Jira project, with no code changing. A metric that moves that far on a scope fix is not measuring delivery. Agree one unit — merged PRs per product change, or Story-level lead time — and publish it for both. Until then neither team's velocity claim is checkable.

3 **Fix the LORA per-change tax directly** · M

It is the true part of the complaint. `BL-9528..9532` are mechanical and ordered; a generator that scaffolds schema, handler, activity migration, authorization entry and FE call from one spec turns six tickets into one plus review. LORA's own people.md designs this as "Tier 0 — a script not a prompt". It is not built.

5 **Write the week-1 curriculum for both** · S EACH

Neither platform has one. LORA has the raw material and the wrong entry point; Bravo has commodity skills and no map of a 498k-LOC monolith.

6 **Re-ask the question when DF2W has volume** · M

DF2W is the cleanest available test of "building a new product on Bravo is easy": fully configured, zero applications, 22 unstarted epics. Its actual cost — first `BLCS` ticket to first production application — is the number this debate needs and does not have.

30, 60, 90 days

Almost none of this is Bravo engineering time — it is engineering management, PMO and, for one item, the LORA squad. That matters, because §5 is the constraint on everything else in this pack. This plan spends about **half a day** of Squad S&U's time.

SEQUENCING RULE

The **staffing decision comes first**, because it sets the capacity every other document's plan assumes.

Days 0–30

SETTLE WHAT IS MEASURED, AND WHO DOES IT

4. **Name the bus factor.** Decide the staffing and the standing allocation for remediation work. *Leadership · a meeting, day 1.* Done when a recorded decision exists, including what percentage of Squad S&U time is available for the other four plans.
1. **Board hygiene.** Wire DF/D2W epics to their `BLCS` children, or say so in the board description. *PMO · 1 d.*
3. **Spike the LORA scaffolding generator.** *LORA squad · 5 d spike.* A prototype generates the `BL-9528..9532` shape.

Days 31–60

AGREE THE UNIT OF COMPARISON

2. **Measure changes, not tickets.** Agree one unit and publish it for both platforms. *EM + LORA squad · 3 d.* Done when "Bravo shipped 40 product changes last month, LORA 44" is a sentence someone can check — and ticket ratios — 1.83× when they favoured LORA, 1.27× when they favoured Bravo — stop being quoted as velocity.

Days 61–90

REMOVE THE TAX, FIX ONBOARDING

3. **Ship the scaffolding generator.** *LORA squad · 15 d.* The strongest argument for Bravo is answered on the merits, not rebutted.
5. **Week-1 curriculum, Bravo** — a map of the monolith and its five product-discrimination mechanisms. *EM + S&U · 3 d.*
5. **Week-1 curriculum, LORA** — invert the entry point from the SDK to a loan moving. *LORA EM · 3 d.*
6. **Start the DF2W clock.** Instrument first-`BLCS`-ticket to first-production-application. *PMO · 1 d.*

Deferred

WITH A TRIGGER

6. **Re-ask the "is Bravo easier" question with real DF2W data.** DF2W has **zero** applications today and 22 unstarted epics — there is nothing to measure yet. *Trigger: when DF2W takes production volume, on the clock started in phase 3.*

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
Delivery tracked where delivery happens	A DF4W epic on board 2099 shows its `BLCS` children and a real start date. The timeline stops reading as nine months of inactivity.
One agreed unit of change	“Bravo shipped 40 product changes last month, LORA shipped 44” is a sentence someone can check, instead of two ticket counts that measure different things.
Scaffolding for the five-repo change	`BL-9528..9532` becomes one ticket and a generated PR set. The strongest argument for Bravo loses most of its force, on the merits.
Bus factor named	Placing a new product family on Bravo is a decision that includes “and we will hire three more engineers” — or it is a decision not taken.
Week-1 curriculum	A Go hire reaches a moving loan in week 1 instead of the SDK; a Java hire reaches a moving loan instead of `SurveyorAssignmentServiceImpl`.
DF2W measured end to end	The claim gets a real number attached to it, on the product it was made about.

> **On velocity, the claim is not supported. On ergonomics, it is right — and the fix is a generator, not a migration.**